Mining Software Repositories

Chapter 1: Introduction and Foundations

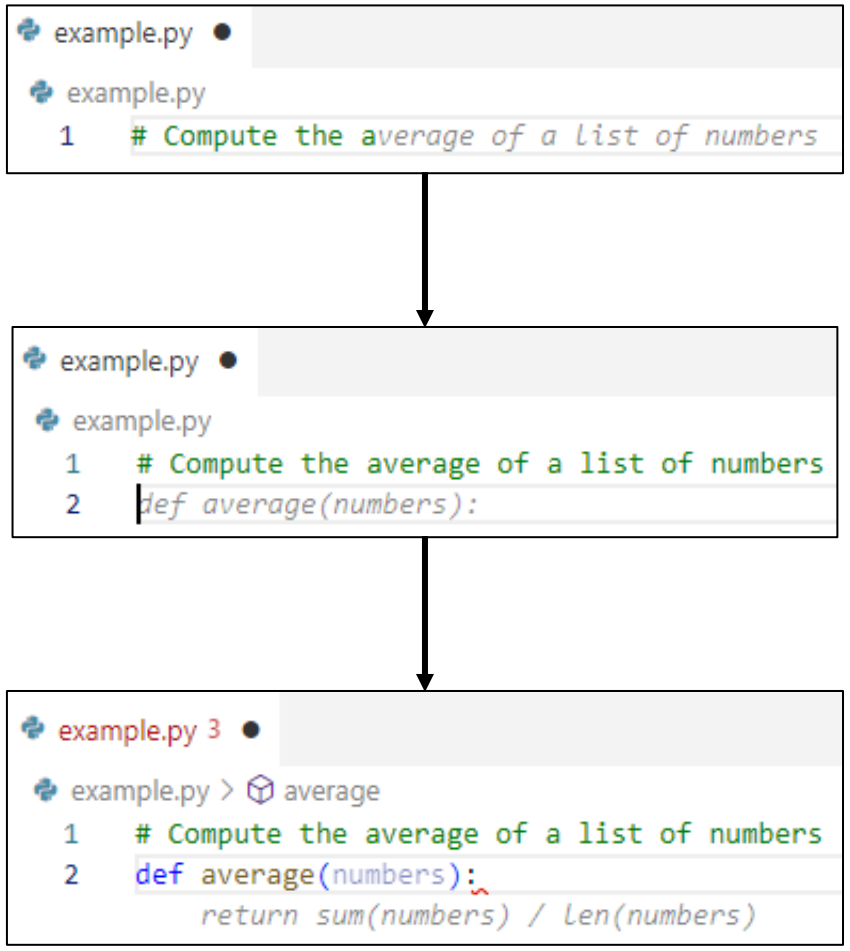
Prof. Dr. Steffen Herbold

Contents

- Motivating examples
- Basic definitions
- Evidence-based software engineering
- Building tools

Motivating examples

GitHub Co-Pilot



Data about code changes is used to train AI models

https://github.com/features/copilot

Insights into development processes



contributed articles

DOI:10.1145/3188720

For a static analysis project to succeed, developers must feel they benefit from and enjoy using it.

BY CAITLIN SADOWSKI, EDWARD AFTANDILIAN, ALEX EAGLE, LIAM MILLER-CUSHON, AND CIERA JASPAN

Lessons from Building Static Analysis Tools at Google

SOFTWARE BUGS COST developers and software companies a great deal of time and money. For example, in 2014, a bug in a widely used SSL implementation ("goto fail") caused it to accept invalid SSL certificates,[36] and a bug related to date formatting caused a large-scale Twitter outage.[23] Such bugs are often statically detectable and are, in fact, obvious upon reading the code or documentation yet still make it into production software.

Previous work has reported on experience applying bug-detection tools to production software.[6,3,7,29] Although there are many such success stories for developers using static analysis tools, there are also reasons engineers do not always use static analysis tools or ignore their warnings,[6,7,26,30] including:

Not integrated. The tool is not integrated into the developer's workflow or takes too long to run;

Not actionable. The warnings are not actionable;

Not trustworthy. Users do not trust the results due to, say, false positives;

Not manifest in practice. The reported bug is theoretically possible, but the problem does not actually manifest in practice;

» key insights

- Static analysis authors should focus on the developer and listen to their feedback.
- Careful developer workflow integration is key for static analysis tool adoption.
- Static analysis tools can scale by crowdsourcing analysis development.

Data can be used to understand the impact of best practices and tools

Social Networks



Gender & Technology CHI 2015, Crossings, Seoul, Korea

Gender and Tenure Diversity in GitHub Teams

Bogdan Vasilescu[†§*], Daryl Posnett[†], Baishakhi Ray[†], Mark G.J. van den Brand[§], Alexander Serebrenik[§], Premkumar Devanbu[†], Vladimir Filkov[†*]
[†]University of California, Davis and [§]Eindhoven University of Technology
[*]vasilescu@ucdavis.edu, filkov@cs.ucdavis.edu

ABSTRACT

Software development is usually a collaborative venture. Open Source Software (OSS) projects are no exception; indeed, by design, the OSS approach can accommodate teams that are more open, geographically distributed, and dynamic than commercial teams. This, we find, leads to OSS teams that are quite diverse. Team diversity, predominantly in offline groups, is known to correlate with team output, mostly with positive effects. How about in OSS?

Using GitHub, the largest publicly available collection of OSS projects, we studied how gender and tenure diversity relate to team productivity and turnover. Using regression modeling of GitHub data and the results of a survey, we show that both gender and tenure diversity are positive and significant predictors of productivity, together explaining a sizable fraction of the data variability. These results can inform decision making on all levels, leading to better outcomes in recruiting and performance.

Author Keywords

Open source; gender; diversity; productivity; GitHub.

ACM Classification Keywords

H.5.3. [Information Interfaces and Presentation (e.g. HCI)]: Computer-supported cooperative work

INTRODUCTION

Because of the world-wide demand for talented and skilled labor, hiring in STEM (Science, Technology, Engineering, and Math) fields has become increasingly almost entirely meritocratic, and largely blind to demographic factors. This is certainly true for software engineering; as a result, both commercial and open source software teams can be very diverse. What are the effects of this on the project as a whole? Indeed, demographic similarity enhances mutual trust (and thus, arguably, team effectiveness), while demographic diversity may lead to stereotyping, cliquishness, and conflict [20, 43]. However, a team's social diversity seems to improve its technical performance [24].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CHI 2015, April 18 - 23, 2015, Seoul, Republic of Korea.
Copyright held by authors. Publication rights licensed to ACM. 978-1-4503-3145-6/15/04$15.00. http://dx.doi.org/10.1145/2702123.2702549

Software development teams can be *diverse* in various ways, *e.g.*, w.r.t. gender, experience, nationality, and coding language preference; some teams can be more diverse in one attribute and less so in others. Diversity attributes may also interact (*e.g.*, in some nations, female professionals may face more obstacles), which complicates analysis and study. Team diversity has been studied in physical ("meat-space") settings; however, data is hard-won in such settings. Smaller sample sizes make it difficult to effectively control for confounds. Data requirements for such effective controls, however, increase exponentially with the number of dimensions studied (one aspect of the "curse of dimensionality" [22]). Thus, studies of effects of diversity in teams (given the ineluctable confounds) require data on a great many teams, with sufficient variance along all co-variates of concern.

GitHub, a social coding platform, has attracted millions of developers and thousands of Open Source Software projects.[1] All commits, issues, code changes, pull-requests etc. are archived and publicly available. GitHub has become the new standard for comprehensive studies of social and technical organization and achievement [16, 37, 39, 41, 60]. Evidently, this is an attractive setting in which to study the relationship of diversity to performance. The scale of GitHub is especially relevant when considering the role of women, who are very underrepresented in programming.[2] With a large enough dataset, however, the effect of increased gender diversity becomes noticeable. Additionally, since all data in GitHub is historical (*i.e.*, archived), it is possible to study the effects of tenure, or one's length of time with a project and with GitHub. However, the reliance on *volunteers* in OSS projects complicates matters; volunteers come and go, leading to team turn-over. Team turnover can certainly influence performance, and will confound the effects of diversity. The constructs of "team" and "team turnover" clearly also depend on the observation time-scale. In a healthy project, some rate of turnover is in fact desirable, as "new blood" brings in new abilities and ideas [21]. Arguably, turnover *will* affect observed diversity in GitHub OSS teams, and must be considered carefully.

In this paper, using GitHub data, we explore several questions: How diverse are online teams with respect to gender and tenure? Does gender diversity depend on tenure? On

[1]OSS depend on distributed volunteers' efforts whereas commercial software is much more centralized, and depends more on paid groups of programmers [23]; in both, the quality can be high [8].
[2]Especially so, it seems, in OSS projects: A 2013 FLOSS Survey [49] indicates 10% females; all earlier surveys [19] agree on merely 1–5%. Industry reports slightly higher numbers, *e.g.*, Google with 17% female technology employees.

Data can be used to understand social aspects of software development

Insights into bugs and failures



2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)

Static source code metrics and static analysis warnings for fine-grained just-in-time defect prediction

Alexander Trautsch
Institute of Computer Science
University of Goettingen, Germany
alexander.trautsch@cs.uni-goettingen.de

Steffen Herbold
Institute AIFB
Karlsruhe Institute of Technology, Germany
steffen.herbold@kit.edu

Jens Grabowski
Institute of Computer Science
University of Goettingen, Germany
grabowski@cs.uni-goettingen.de

***Abstract*—Software quality evolution and predictive models to support decisions about resource distribution in software quality assurance tasks are an important part of software engineering research. Recently, a fine-grained just-in-time defect prediction approach was proposed which has the ability to find bug-inducing files within changes instead of only complete changes. In this work, we utilize this approach and improve it in multiple places: data collection, labeling and features. We include manually validated issue types, an improved SZZ algorithm which discards comments, whitespaces and refactorings. Additionally, we include static source code metrics as well as static analysis warnings and warning density derived metrics as features. To assess whether we can save cost we incorporate a specialized defect prediction cost model. To evaluate our proposed improvements of the fine-grained just-in-time defect prediction approach we conduct a case study that encompasses 38 Java projects, 492,241 file changes in 73,598 commits and spans 15 years. We find that static source code metrics and static analysis warnings are correlated with bugs and that they can improve the quality and cost saving potential of just-in-time defect prediction models.**

***Index Terms*—Software quality, Software metrics**

I. Introduction

Quality assurance budgets are limited. A risk analysis for changes introduced to software would provide hints for quality assurance personal on how to make the most of their limited resources. Just-in-time defect prediction models are predictive models that assign a risk to changes, or files within a change, of being defect-inducing. Because just-in-time models are able to provide feedback directly after a change happened, they can reduce the cost of bug removal.

Just-in-time defect prediction is an active research topic which tries to enable the aforementioned theoretical risk probabilities on a per-change basis. A lot of research is being conducted in this area, e.g., improving the granularity of the predictions [1], adding features, e.g., code review [2], change context [3], or applying deep learning models [4].

Just-in-time defect prediction models are trained on bug-inducing changes, which are found by tracing back bug-fixing changes, e.g., with the SZZ algorithm [5]. Some researchers utilize keyword only approaches that scan the commit messages for certain keywords, e.g., fixes, fixed or patch, to find bug-fixing commits, e.g., [1, 3, 6]. We refer to this approach as ad-hoc SZZ. Others apply full SZZ which requires a link from the commit to the Issue Tracking System (ITS) for identifying bug-fixing commits, e.g., [2, 7, 8]. We refer to this approach as ITS SZZ. In addition to this information, features that describe the change are used as independent variables, e.g., size of the change or diffusion, i.e., how may different subsystems are changed [1, 7, 9].

In contrast to just-in-time defect prediction, release-level defect prediction utilizes features describing files, classes or methods. These features encompass size and complexity metrics as well as object oriented metrics extracted from the files. D'Ambros et al. [10] incorporated change level features into release level defect prediction by including a time-frame before the release for change metric calculation. In addition to static, size and complexity metrics, static analysis warnings were also investigated in the context of quality insurance. Rahman et al. [11] investigated static analysis warnings in the context of release-level defect prediction. Zheng et al. [12] found that the number of static analysis warnings may help to identify defective modules.

Static analysis warnings are generated by tools which inspect different source code representations, e.g., Abstract Syntax Trees (ASTs) or call graphs and find patterns that are known to be problematic. If a problematic pattern is found a warning for the line in the code is generated for the developer. The combination of pattern and generated warning is defined in a rule, these tools allow the developers to define which rules should be utilized by the tool. The rules depend on the tool but most are concerned with readability, common coding mistakes as well as size and complexity thresholds. Results of questionnaires show that developers assign importance to static analysis software, especially at code review time [13], see also Panichella et al. [14].

Recently, Pascarella et al. [1] introduced a fine-grained just-in-time defect prediction approach where instead of complete changes the files contained in these changes are used to train predictive models. Static analysis warnings were not included, however the authors adopted change features for their fine-grained approach together with an ad-hoc SZZ implementation. While Querel et al. [15] included static analysis warnings

2576-3148/20/$31.00 ©2020 IEEE
DOI 10.1109/ICSME46990.2020.00022

Authorized licensed use limited to: University of Passau. Downloaded on March 08,2023 at 14:09:52 UTC from IEEE Xplore. Restrictions apply.

Data can be used to understand how to avoid or find problems

Basic definitions

Software repositories

- Definition following ISO/IEC/IEEE 24765:2017:
  - collection of all system element or software-related artifacts belonging to a system
  - location/format in which such a collection is stored
  - organized and persistent data storage that allows data retrieval

- From the homepage of the 23rd International Conference on Mining Software Repositories:
  - "software data such as source control systems, defect tracking systems, code review repositories, archived communications between project personnel, question-and-answer sites, CI build servers, and run-time telemetry.”

- Kagdi et al., 2007:
  - “artifacts that are produced and archived during software evolution”

https://conf.researchr.org/home/msr-2023

Kagdi, H., Collard, M. L., & Maletic, J. I. (2007). A survey and taxonomy of approaches for mining software repositories in the context of software evolution. *Journal of software maintenance and evolution: Research and practice*, *19*(2), 77-131.

Many different kinds of repositories!

Continuous integration systems



circleci



Issue tracker



Jira

Q&A sites

stackoverflow



git

Source code management and version control

Forges that combine many aspects

sourceforge

GitLab

GitHub

Code review tools



Gerrit

slack



Mattermost

Developer chats

… and more!

Data mining

- Meriam-Webster:
  - the practice of searching through large amounts of computerized data to find useful patterns or trends

- Key aspects of this definition:
  - Large amounts of computerized data
  - Searching
  - Finding useful patterns or trends

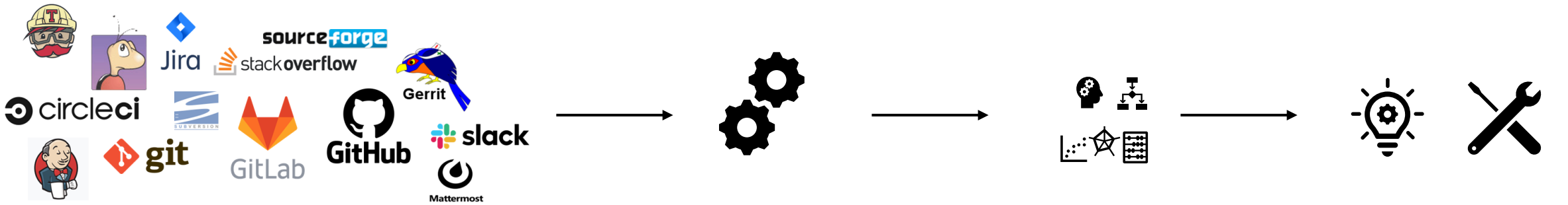
- Analogy with mining of physical resources:
  - Raw data is similar to raw metals in form of ores
  - Searching within that data is similar to retrieving ores from a mine
  - Finding useful patterns is similar to refining ores into metals



Data mining techniques

- Term data mining is used very generically
  - Similar to data science
- Usually used as broad category for analyzing data
- Data retrieval depends on problem
  - Can be automated
    - e.g., from a warehouse
  - Can be manual
    - e.g., through manual coding of data
  - Can be semi-automated
    - e.g., through a human-guided interactive search or through the human augmentation of automatically collected data
- Data analysis not restricted to specific techniques, but often
  - automated analysis of data through machine learning (both supervised and unsupervised)
  - statistical analysis of data (e.g., distributions, statistical tests)
  - anomaly detection
  - dimensionality reduction
  - visualization

Software repositories + data mining = Mining Software Repositories (MSR)



Data from software repositories → Data collection → Data analysis → Insights or tools

Evidence-based software engineering

Example: Evidence-based medicine

- Standard in conventional medicine:
  - Suggest a therapy
    - E.g., a new drug
  - Collect evidence regarding how it works
    - Does the therapy have the desired positive effect?
    - Does the therapy have other positive side-effects?
    - Does the therapy have negative side-effects?
    - How likely is each of the above?
  - Decisions on how therapy may be used is based on this evidence
  - All of the above is (hopefully) based on rigorous study designs with the goal to demonstrate causal relationships between therapies and outcomes
  - Often a multi-stage process, e.g.:
    - Lab experiments (in vitro)
    - Animal experiments (in vivo)
    - Initial study with human subjects for safety analysis (small sample)
    - Randomized trail with many subjects to understand effects of treatments (large sample with treatment and a control group without treatment)

→ Evidence-based decision making and evidence-based medicine

And software engineering?

Usually someone suggests something

Some group discusses this

And then a decision is made

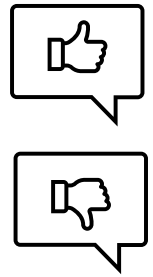
Introduction

This PEP contains the index of all Python Enhancement Proposals, known as PEPs. PEP numbers are assigned by the PEP editors, and once assigned are never changed. The version control history of the PEP texts represent their historical record.

Other Informational PEPs

	PEP	Title	Authors
IA	20	The Zen of Python	Peters
IA	101	Doing Python Releases 101	Warsaw, GvR
IF	247	API for Cryptographic Hash Functions	Kuchling
IF	248	Python Database API Specification v1.0	Stein, Lemburg





Example: PEPs to guide Python development

Not guided by rigorous evidence, but rather experience and anecdotal evidence

https://peps.python.org/

Evidence-based software engineering

When someone suggests something

We collect evidence about the suggestion

And then a decision is made

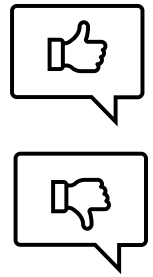
Introduction

This PEP contains the index of all Python Enhancement Proposals, known as PEPs. PEP numbers are assigned by the PEP editors, and once assigned are never changed. The version control history of the PEP texts represent their historical record.

Other Informational PEPs

	PEP	Title	Authors
IA	20	The Zen of Python	Peters
IA	101	Doing Python Releases 101	Warsaw, GvR
IF	247	API for Cryptographic Hash Functions	Kuchling
IF	248	Python Database API Specification v1.0	Stein, Lemburg



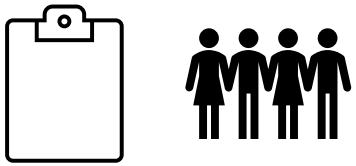


Guided by evidence, but requires a lot more effort!

How to collect evidence?

C. Wohlin, P. Runeson, M. Höst, M.C. Ohlsson, B. Regnell, and A. Wesslén. Experimentation in Software Engineering. Springer Berlin Heidelberg, 2012. ISBN 9783642290435

Survey



A survey is a system for collecting information from or about people to describe, compare or explain their knowledge, attitudes and behavior

Case study



Case study in software engineering is: an empirical enquiry that draws on multiple sources of evidence to investigate one instance (or a small number of instances) of a contemporary software engineering phenomenon within its real-life context, especially when the boundary between phenomenon and context cannot be clearly specified

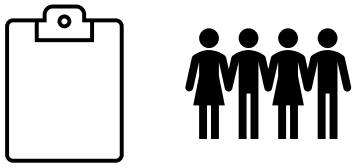
Experiment



Experiment (or controlled experiment) in software engineering is an empirical enquiry that manipulates one factor or variable of the studied setting. Based in randomization, different treatments are applied to or by different subjects, while keeping other variables constant, and measuring the effects on outcome variables. In human-oriented experiments, humans apply different treatments to objects, while in technology-oriented experiments, different technical treatments are applied to different objects

MSR for evidence-based software engineering

Survey



- Repositories can be used to create developer profiles
  - Based on their contributions, e.g., commits, issues, emails, etc.
  - Within this lecture, when we speak of developer, we usually mean that in a broad sense as anybody somehow involved in a project. Thus, writing code is not a requirement, other activities like issue reporting, or being part of email or chat conversations is also sufficient. If we instead want to refer to someone actually writing code, we will make that explicit.
- Helps to identify suitable subjects, e.g., developers that fit a certain profile.
  - Familiar with a programming language
  - Familiar with a technology (e.g., a framework)
  - Core developer of a project
  - …
- Not without ethical issues and risks
  - Repeated requests for survey participation may lead to problems
  - Happened when GHtorrent was repeatedly used to contact developers
  - Any study that involves personal data needs to check if an ethics approval is required
- Even with ethical approach, data protection regulations my still apply!

https://gousios.org/blog/Issue-thirty-two.html

MSR for evidence-based software engineering

Case study



Experiment



- Software repositories contain huge amounts of data
- This data can be used for many research purposes
  - Want to study code reviews? You can view them on GitHub, Gerrit, etc.
  - Want to study code quality? Code is available in the source code versioning system, information about defects in issue trackers.
  - Want to study how developers in different time-zones interact? Who contributes to the same parts of a software can be reconstructed from repositories.
  - …
- Can be used both for case studies and experiments
  - Depends on the design!
  - Are you only observing a group of subjects to draw conclusions → case study
  - Are you looking at different treatment groups are studying interventions → experiment

Building tools

Tools support for software development

- We use tools to aid all aspects of software engineering
  - IDEs for writing code
  - Testing tools and code review tools for quality assurance
  - Issue trackers for requirements and change management
  - …
- MSR can help improve these tools, e.g.,
  - Rank static analysis warnings based on past fixes of warnings
  - Provide a risk analysis based on past bugs to guide quality assurance efforts
  - Identify which developers are well-suited to work on an issue based on past contributions
  - Identify potential reviewers for pull requests
  - Aid issue triage by automatically determining the affected components and/or issue type
  - …

Usual MSR tool building approach

- Identify a goal for the tool
  - Which part of software development do you want to help?
  - How do you think your tool will be used?
- Identify which data you need
  - Can the data be automatically obtained? How? From where?
  - Does the data need to be updated frequently to avoid concept drift?
  - Do you require regular manual interventions (e.g., in form of labeling) to collect data?
  - How noisy is the data? Will it be good enough?
- Collect data and build tool
- Collect more data and conduct an experiment

→MSR tool building is usually combined with evidence regarding how the tools work!

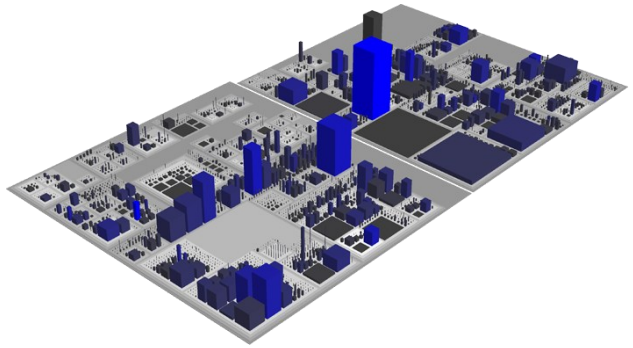
… but the evidence if often not a business metric (ROI) or even user metric (usability, time saved) but a proxy (e.g., accuracy)

MSR tools and machine learning

- Many MSR tools rely on machine learning
  - Collect data from some repositories
  - Build machine learning model
  - Evaluate model on other repositories
  - Determine if/how this would help developers

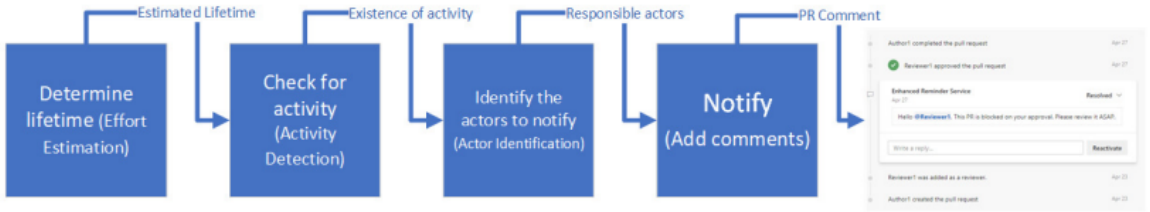
- But there are also tools without machine learning
  - Visualize information from repositories
    - For example, to show how parts of the software interact with each other or how large the software is
  - Aggregate information from different repositories into a single repository
    - For example, as overlay to understand how many open issues there are when multiple issue trackers are used
  - Analysis of repository contents with standard software engineering techniques, e.g., to detect clones or potential flaws on a large scale

Examples for MSR tools

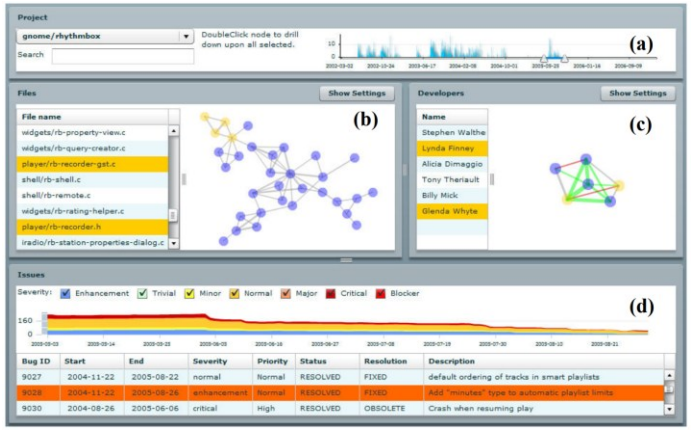


Code City to visualize the source code size, complexity, and package structure

Nudge: Accelerating Overdue Pull Requests Towards Completion



Nudge to Aid developers to complete tasks



Tesseract to analyze who works together on what

Wettel, R., Lanza, M., & Robbes, R. Software systems as cities: A controlled experiment. In *Proceedings of the 33rd International Conference on Software Engineering* (pp. 551-560), 2011

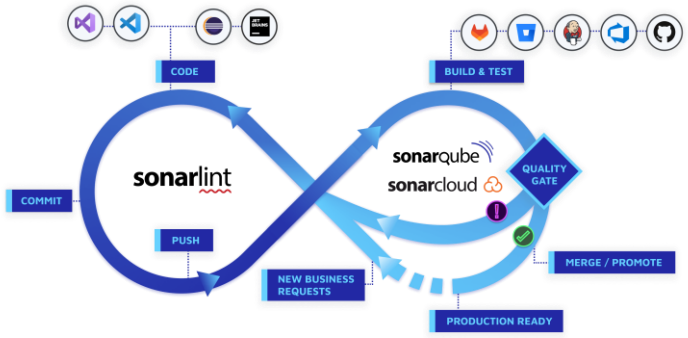
Chandra Maddila, Sai Surya Upadrasta, Chetan Bansal, Nachiappan Nagappan, Georgios Gousios, and Arie van Deursen. 2022. Nudge: Accelerating Overdue Pull Requests Towards Completion. ACM Trans. Softw. Eng. Methodol. Just Accepted (June 2022). https://doi.org/10.1145/3544791

Sarma, A., Maccherone, L., Wagstrom, P., & Herbsleb, J. (2009, May). Tesseract: Interactive visual exploration of socio-technical relationships in software development. In *2009 IEEE 31st International Conference on Software Engineering* (pp. 23-33). IEEE.

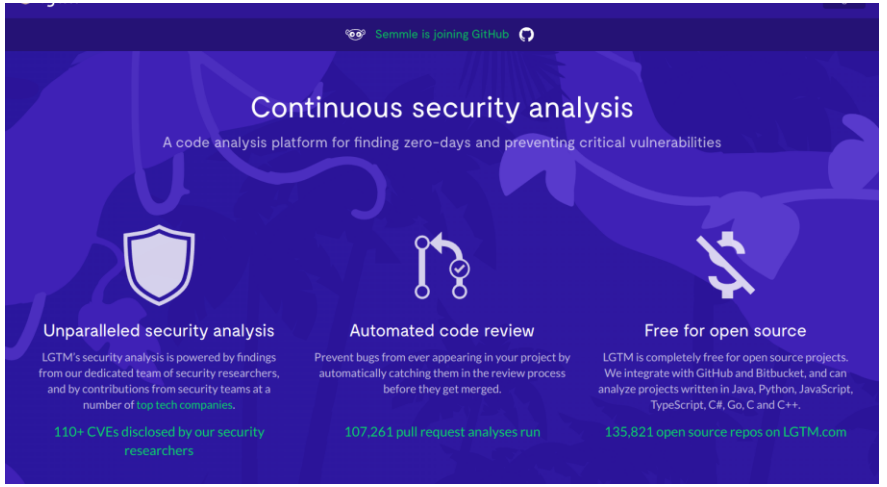
MSR as a Product

There are companies that specialize in the analysis of code repositories

CQSE Teamscale



Others were acquired and integrated into larger products



Such tools combine analysis with integration into a repository/company ecosystem

Questions?

